

Лабораторная работа №8

Оптимизация

Доберштейн А. С.

Российский университет дружбы народов, Москва, Россия

Информация

- Доберштейн Алина Сергеевна
- НФИбд-02-22
- Российский университет дружбы народов

Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

Выполнение лабораторной работы

Повторим примеры из лабораторной работы.

```
[1]: # Изменение namespace:
import Pkg
Pkg.add("JuMP")
Pkg.add("GLPK")
using JuMP
using GLPK

#[32m[1m Resolving[22m[39m package versions... ***

[2]: # Определяем область модели с именем model:
model = Model{GLPK.Optimizer}

[2]: A JuMP Model
  solver: GLPK
  objective sense: FEASIBILITY_SENSE
  num_variables: 0
  num_constraints: 0
  Names registered in the model: none

[3]: # Определяем переменные x, y и граничные условия для них:
@variable(model, x >= 0)
@variable(model, y >= 0)

[3]: y

[4]: # Определяем целевую функцию:
@constraint(model, 7x + 12y >= 120)
@constraint(model, 7x + 12y >= 120)

[4]: 7x + 12y ≥ 120
```

Рис. 1: Линейное программирование

Выполнение лабораторной работы

```
[5]: # Определение целевой функции:
    objective(model, Min, 12x + 20y)

[5]: 12x + 20y

[6]: # Выход функции оптимизации:
    optimize!(model)

[7]: # Определение причины завершения работы оптимизатора:
    termination_status(model)

[7]: OPTIMAL::TerminationStatusCode = 1

[8]: # Демонстрируем приближенные результирующие значения переменных x и y:
    @show value(x);
    @show value(y);
    # Демонстрируем результаты оптимизации:
    @show objective_value(model);

value(x) = 14.090909090909091
value(y) = 1.1500000000000007
objective_value(model) = 205.0
```

Рис. 2: Линейное программирование

Выполнение лабораторной работы

```
[9]: #Инициализация rosbabel:  
import pkg  
pkg.add("JuMP")  
pkg.add("GLPK")  
using JuMP  
using GLPK  
  
#(52m)[1m Resolving[(22m)[30m package versions...***  
  
[10]: #Определение объектов модели: c - целевая функция, vector_model:  
vector_model = Model{GLPK.Optimizer}  
  
[10]: A JuMP Model  
├ solver: GLPK  
├ objective_sense: FEASIBILITY_SENSE  
├ num_variables: 0  
├ num_constraints: 0  
└ Names registered in the model: none  
  
[11]: #Определение начальных данных:  
A = [ 1 1 0 0;  
      1 0 0 0;  
      2 0 0 1];  
b = [10; 5]  
c = [1; 1; 1; 1]  
  
[12]: 4-element Vector{Int64}:  
 1  
 3  
 5  
 2
```

Рис. 3: Векторизованные ограничения и целевая функция оптимизации

Выполнение лабораторной работы

```
[14]: #Определение базиса переменных:
      @variable(vector_model,x[i:i] >=0)

[14]: 4-element Vector{VariableRef}:
      x[1]
      x[2]
      x[3]
      x[4]

[15]: #Определение ограничений модели:
      @constraint(vector_model,A * x .>=b)

[15]: 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
      x[1] + x[2] + 3 x[3] + 5 x[4] = 7
      3 x[1] + 5 x[2] + 8 x[3] = 3
      2 x[1] + 6 x[3] + 13 x[4] = 5

[16]: # Определение целевой функции:
      @objective(vector_model,MIO,c' *x)

[16]: 
$$x_1 + 3x_2 + 5x_3 + 2x_4$$


[16]: Функция оптимизации:
      optimize!(vector_model)

[19]: # Определение причины завершения работы оптимизатора:
      termination_status(vector_model)

[19]: OPTIMAL::TerminationStatusCode = 1

[20]: # Демонстрация результатов оптимизации:
      @show objective_value(vector_model);

      objective_value(vector_model) = 4.8238769238769225
```

Рис. 4: Векторизованные ограничения и целевая функция оптимизации

Выполнение лабораторной работы

```
[21]: # Добавление пакетов:
import Plg
Plg.add("JuMP")
Plg.add("GLPK")
using JuMP
using GLPK

0[10m][1m Resolving[21m][2m package versions...***

[22]: # Контейнер для хранения данных об ограничениях на количество потребления мяса, хлеба, яиц и соли:
category_data = JuMP.Containers.DenseAxisArray{
  [1800, 2200],
  91 Int,
  0 0},
  0 1770],
  ["calories", "protein", "fat", "sodium"],
  ["min", "max"]}

# Массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]

# Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray{
  [2.49, 2.89, 1.59, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59], foods}

[22]: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
  And data, a 9-element Vector{Float64}:
  2.49
  2.89
  1.5
  1.89
  2.09
  1.99
  2.49
  0.89
  1.59
```

Рис. 5: Оптимизация рациона питания

Выполнение лабораторной работы

```
[23]: # Массив данных о содержании калорий, белков, жиров и солей в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray{
[410 26 26 730;
 420 32 10 1100;
 500 20 12 1000;
 500 4 19 270;
 320 13 10 930;
 520 15 12 820;
 320 31 12 1230;
 100 0 2.5 125;
 330 0 10 180]}.
foods,
["calories", "protein", "fat", "sodium"])

[23]: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
  Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9x4 Matrix{Float64}:
410.0  26.0  26.0  730.0
420.0  32.0  10.0  1100.0
500.0  20.0  12.0  1000.0
500.0   4.0  19.0  270.0
320.0  12.0  10.0  930.0
520.0  15.0  12.0  820.0
320.0  31.0  12.0  1230.0
100.0   0.0   2.5  125.0
330.0   0.0  10.0  180.0

[24]: # Определяем объектную модель с помощью model:
model = Model{GLPK.Optimizer}

[24]: A JuMP Model
 | solver: GLPK
 | objective_sense: FEASIBILITY_SENSE
 | num_variables: 0
 | num_constraints: 0
 | Names registered in the model: none
```

Рис. 6: Оптимизация рациона питания

Выполнение лабораторной работы

```
[25]: # Objective method:
categories = ["calories", "protein", "fat", "sodium"]

[25]: 4-element Vector{String}:
 "calories"
 "protein"
 "fat"
 "sodium"

[26]: # Objective expression:
@variables(model, begin
  category_data[c, "min"] <- nutrition[c = categories] <=
  category_data[c, "max"]
# Common objective expression:
buy[foods] >= 0
end)

[26]: (1-dimensional DenseVector{VariableRef,1,...}) with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{VariableRef}:
 nutrition[calories]
 nutrition[protein]
 nutrition[fat]
 nutrition[sodium], 1-dimensional DenseVector{VariableRef,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 8-element Vector{VariableRef}:
 buy[hamburger]
 buy[chicken]
 buy[hot dog]
 buy[fries]
 buy[macaroni]
 buy[pizza]
 buy[salad]
 buy[milk]
 buy[ice cream])
```

Рис. 7: Оптимизация рациона питания

Выполнение лабораторной работы

```
[27]: # Определение целевой функции:
      objective(model, Min, sum(cost[f] * buy[f] for f in foods))

[27]: 2.49buy_hamburger + 2.89buy_chicken + 1.5buy_hot_dog + 1.89buy_fries + 2.09buy_macaroni + 1.99buy_pizza + 2.49buy_salad + 0.89buy_milk +
      0.99buy_ice_cream

[28]: # Определение ограничений модели:
      @constraint(model, [c in categories], sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])

[28]: 1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.EqualTo{Float64}}, ScalarShape{1},...} with index sets:
      Dimension 1, ["calories", "protein", "fat", "sodium"]
      And data, = 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.EqualTo{Float64}}, ScalarShape{1}}:
      -nutrition[calories] + 410 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] + 320 buy[pizza] + 320 buy[salad] + 180 buy[milk] + 330 buy[ice cream] = 0
      -nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza] + 31 buy[salad] + 8 buy[milk] + 8 buy[ice cream] = 0
      -nutrition[fat] + 26 buy[hamburger] + 10 buy[chicken] + 32 buy[hot dog] + 19 buy[fries] + 10 buy[macaroni] + 12 buy[pizza] + 11 buy[salad] + 2.5 buy[milk] + 10 buy[ice cream] = 0
      -nutrition[sodium] + 220 buy[hamburger] + 1100 buy[chicken] + 1000 buy[hot dog] + 270 buy[fries] + 930 buy[macaroni] + 820 buy[pizza] + 1230 buy[salad] + 125 buy[milk] + 180 buy[ice cream] = 0

[29]: # Поиск оптимального решения:
      JuMP.optimize!(model)
      term_status = JuMP.termination_status(model)

[29]: OPTIMAL::TerminationStatusCode = 1

[30]: best(buy_data, JuMP.value.(buy_data))

[30]: 9x2 Matrix{AffExpr}:
      buy[hamburger]  0.6045158888888888
      buy[chicken]   0
      buy[hot dog]   0
      buy[fries]     0
      buy[macaroni]  0
      buy[pizza]     0
      buy[salad]     0
      buy[milk]      6.9701388888888935
      buy[ice cream] 2.5932194444444441
```

Рис. 8: Оптимизация рациона питания

Выполнение лабораторной работы

```
[11]: # Subsequent rows:
import Pkg
Pkg.add("DelimitedFiles")
Pkg.add("CSV")
using DelimitedFiles
using CSV

R[120] is Resolving R[120] 3w package versions...***

[12]: # Create our dataset:
passportdata = readall(joinpath("passport-index-matrix.csv"),';',';')

[13]: 200x200 Matrix{Any}:
"Passport"      "Albania"      - "Afghanistan"
"Afghanistan"   "visa required" -1
"Albania"       -1              "visa required"
"Algeria"       "visa required" "visa required"
"Andorra"       90              "visa required"
"Angola"        "visa required" "visa required"
"Antigua and Barbuda" 90              "visa required"
"Argentina"     90              "visa required"
"Armenia"       90              "visa required"
"Australia"     90              "visa required"
"Austria"       90              "visa required"
"Azerbaijan"    90              "visa required"
"Bahamas"       90              "visa required"
⋮
"United Arab Emirates" 90              "visa required"
"United Kingdom"      90              "visa required"
"United States"       90              "visa required"
"Uruguay"             90              "visa required"
"Uzbekistan"          "visa required" "visa required"
"Vanuatu"             "visa required" "visa required"
"Vatican"             90              "visa required"
"Venezuela"          90              "visa required"
"Vietnam"            "visa required" "visa required"
"Yemen"              "visa required" "visa required"
"Zambia"             "visa required" "visa required"
"Zimbabwe"           "visa required" "visa required"
```

Рис. 9: Путешествие по миру

[illegible]

Рис. 10: Путешествие по миру

Выполнение лабораторной работы

```
[44]: # Завершение оптимизации:
JuMP.optimize!(model)
termination_status(model)

[44]: OPTIMAL::TerminationStatusCode = 1

[45]: # Вывод результатов:
print(JuMP.objective_value(model), " passports:", join(ctr{findall(JuMP.value.(pass) .== 1}}, ", "))

61.0 passports:Afghanistan, Andorra, Argentina, Australia, Azerbaijan, Bahrain, Brunei, Cambodia, Cameroon, Canada,
Chile, Colombia, Comoros, DR Congo, Djibouti, Equatorial Guinea, Eritrea, Fiji, Gabon, Georgia, Guinea, Guinea-Bissau,
Hong Kong, Hungary, Indonesia, Iraq, Ireland, Israel, Jamaica, Japan, Kuwait, Laos, Liberia, Libya, Macao, Madagascar,
Malaysia, Maldives, Marshall Islands, Mauritania, Mauritius, Mongolia, Mozambique, Myanmar, Nepal, New Zealand, North Korea,
Palestine, Papua New Guinea, Qatar, Saudi Arabia, Solomon Islands, Somalia, South Sudan, Sri Lanka, Syria, Taiwan,
Timor-Leste, Togo, Turkmenistan, United States, Uruguay, Vietnam
```

Рис. 11: Путешествие по миру

Выполнение лабораторной работы

```
•[64]: # Добавление необходимых пакетов:
import Pkg
Pkg.add("DataFrames")
Pkg.add("XLWX")
Pkg.add("Plots")
Pkg.add("PyPlot")
Pkg.add("Random")
Pkg.add("GCS")
Pkg.add("Statistics")

@[12m]Im Resolving@[22m]3m package versions... ***

[50]: # Читаем данные в формате из файла:
T = DataFrames.XLWX.readtable("stock_prices.xlsx", "Sheet2")

[50]: 13×3 DataFrame
      Row  MSFT  FB  AAPL
      Any  Any  Any
1  101.93  137.05  148.26
2  102.8   141.8   152.29
3  107.71  150.04  156.82
4  107.17  149.01  157.78
5  102.78  165.71  166.52
6  105.67  167.33  170.41
7  108.22  162.5   170.42
8  110.97  161.89  172.97
9  112.53  162.28  174.97
10 110.51  169.6   172.91
11 115.01  165.98  186.12
12 117.05  164.34  191.05
13 117.94  166.69  189.95
```

Рис. 12: Портфельные инвестиции

Выполнение лабораторной работы

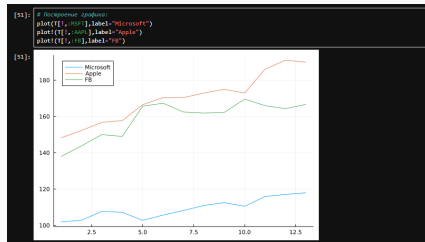


Рис. 13: Портфельные инвестиции

Выполнение лабораторной работы

```
[52]: # Данные о ценах на акции размещаем в матрице:
prices_matrix = Matrix(T)

[52]: 12x3 Matrix[Any]:
101.93 137.95 140.26
102.8 145.8 152.29
107.71 150.04 156.82
107.17 149.03 157.76
102.78 165.71 166.52
105.87 167.33 170.41
108.22 162.5 170.42
110.97 161.89 172.97
112.53 162.28 174.97
110.21 169.6 172.91
115.91 165.98 186.12
117.05 164.34 191.05
117.94 166.69 189.95

[53]: # Вычисляем матрицу доходности за период времени:
M1 = prices_matrix[1:end-1,:]
M2 = prices_matrix[2:end,:]
# Матрица доходности:
R = (M2 - M1) ./ M1

[53]: 12x3 Matrix{Float64}:
 0.00932522  0.0624007  0.027182
 0.0477626  0.0633936  0.0297409
-0.00501340 -0.00666484  0.00599413
-0.040963  0.112873  0.055524
 0.628183  0.00977611  0.0233606
 0.0241317 -0.02086051  5.8602e-5
 0.0254112 -0.00375385  0.014963
 0.0140579  0.00240904  0.0115627
-0.0179608  0.0651072 -0.0117754
 0.0408044 -0.0215445  0.0703901
 0.00983522 -0.00980871  0.0204883
 0.00740355  0.0142996 -0.00575166
```

Рис. 14: Портфельные инвестиции

Выполнение лабораторной работы

```
[93]: using DataFrames
      using ALSA
      using Plots
      using Convex
      using SCS
      using Statistics

[72]: # Опасно: рискод:
      risk_matrix = cov(R)

[72]: 3x3 Matrix{Float64}:
      0.0008591813  -0.0006380553  0.0001591112
      -0.0006380553  0.00152162  0.0001922288
      0.0001591112  0.0001922288  0.000635505

[73]: # Проверка положительной определенности матрицы рискод:
      isposdef(risk_matrix)

[73]: true

[74]: # Опасно: опасно: опасно:
      r = mean(R, dim=1)[1:]

[74]: 3-element Vector{Float64}:
      0.812532748740126596
      0.8105038105038105
      0.8211458804655032917

[75]: # Опасно: опасно:
      x = Variable(length(r))

[75]: Variable
      size: (3, 1)
      sign: real
      vexity: offline
      id: 140...370
```

Рис. 15: Портфельные инвестиции

Выполнение лабораторной работы

```
[76]: # Объект модели:
problem = minimize(Convex.quadform(x,risk_matrix),[sum(x)==1;r'*x==0.02;x.>=0])

# Находим решение:
solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 11.44 seconds, 826.956 MiB of memory allocated

-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 5, constraints m: 12
cones:    z: primal zero / dual free vars: 1
          l: linear vars: 4
          q: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-and-qdldl
          nnz(A): 22, nnz(P): 0

-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
0 | 1.45e+01 | 1.00e+00 | 2.01e+01 | -9.96e+00 | 1.00e-01 | 3.60e-03
75 | 1.70e-04 | 3.73e-05 | 5.25e-06 | 4.62e-04 | 1.00e-01 | 1.24e-02
-----
status: solved
timings: total: 1.24e-02s = setup: 1.27e-04s + solve: 1.23e-02s
        lin-sys: 4.85e-05s, cones: 3.10e-05s, accel: 1.00e-05s
-----
objective = 0.000462
-----

[76]: Problem statistics
      problem is DCP          : true
      number of variables    : 1 (3 scalar elements)
      number of constraints   : 5 (5 scalar elements)
      number of coefficients  : 19
      number of atoms         : 14

Solution summary
  termination status : OPTIMAL
  primal status      : FEASIBLE_POINT
  dual status        : FEASIBLE_POINT
  objective value     : 0.0005

Expression graph
  x[1]
  x[2]
  x[3]
  x[4]
  x[5]
```

Рис. 16: Портфельные инвестиции

```
[78]: x
[78]: Variable
      size: (5, 1)
      sign: real
      const: affine
      id: 549_570
      value: [0.07740709795031134, 0.11606771003983090, 0.8065276266877964]

[79]: sum(x.value)
[79]: 1.0000024346779448

[80]: r**x.value
[80]: 1x1 adjoint({Vector(Float64)}) with eltype Float64:
      0.0196472331085319

[81]: x.value .* 1000
[81]: 3x1 Matrix{Float64}:
      77.40709795031134
      116.06771003983090
      806.5276266877964
```

Рис. 17: Портфельные инвестиции

Выполнение лабораторной работы

```
[84]: # Подключение необходимых пакетов:
import Pkg
Pkg.add("ImageMagick")
Pkg.add("Images")
Pkg.add("Convex")
Pkg.add("SCS")
using Images
using Convex
using SCS

@[32m@]m Resolving@[22m@]m package versions...***

[85]: using Images
      using Convex
      using SCS

[86]: # Считывание исходного изображения:
Kret = load("kret-mall.jpg")

[86]: 
```

Рис. 18: Восстановление изображения

Выполнение лабораторной работы

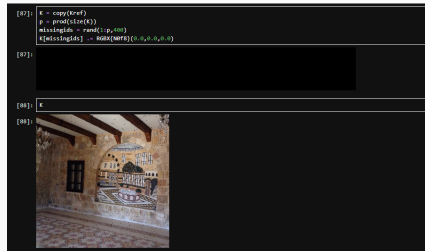


Рис. 19: Восстановление изображения

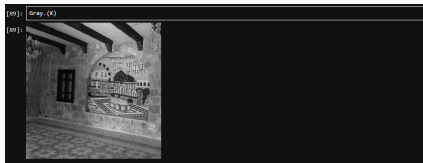


Рис. 20: Восстановление изображения

Выполнение лабораторной работы

```
[90]: # Minimize offset()
      Y = float64.Gray(K);

[91]: correctids = findall(Y[-1,:])
      X = Convex.Variable(nsize(Y))
      problem = minimize(nuclearnorm(X))
      problem.constraints += X[correctids]==Y[correctids]

[92]: Element Vector[constraint]:
      == constraint (affine)
      L = affine; real
      | = index (affine); real
      | = 283x283 real variable (id: 381.639)
      L = sparse Matrix(float64)

[93]: # Maximize psnrmax:
      solve!(problem, GCS.Optimizer)

[ Info: [convex.jl] Compilation finished: 5.05 seconds, 436.000 MiB of memory allocated
-----
GCS v0.2.0 - Splitting Convex Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 80800, constraints m: 159781
cones:
:: primal zero / dual free vars: 76601
nuc: nuclear vars: 80800, nucsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
max_iters: 100000, normalizer: 1, rho_x: 1.00e-06
acceleration_lookback: 10, acceleration_interval: 10
compiled with openmp parallelization enabled
lin-sys: sparse-direct-mmd-qdldl
nnz(A): 159781, nnz(P): 0
-----
iter | pri res | dual res | gap | obj | scale | time (s)
-----
0 | 1.27e+02 | 1.17e+01 | 1.30e+03 | 1.30e+03 | 3.00e+01 |
150 | 4.10e-03 | 6.10e-02 | 1.11e-04 | 4.45e+02 | 6.20e-03 | 2.91e+01
-----
status: solved
timing: total: 2.81e+01s = setup: 2.54e-01s + solve: 1.99e+01s
```

Рис. 21: Восстановление изображения

Выполнение лабораторной работы



Рис. 22: Восстановление изображения

Выполнение лабораторной работы

```
[100]: # 1. Линейное программирование
      model = Model(GLPK.Optimizer)

[100]: A JuMP Model
      | solver: GLPK
      | objective_sense: FEASIBILITY_SENSE
      | num_variables: 0
      | num_constraints: 0
      | Names registered in the model: none

[101]: @variable(model, 0<=x1<=10)
      @variable(model, x2>=0)
      @variable(model, x3>=0)

      @constraint(model, -x1+x2+3x3 <= -5)
      @constraint(model, x1+3x2-7x3 <= 10)

      @objective(model, Max, x1+2x2+5x3)

[101]:  $x_1 + 2x_2 + 5x_3$ 

[102]: optimize!(model)

      termination_status(model)

[102]: OPTIMAL::TerminationStatusCode = 1

[103]: @show value(x1);
      @show value(x2);
      @show value(x3);

      @show objective_value(model);

      value(x1) = 10.0
      value(x2) = 2.1875
      value(x3) = 0.9375
      objective_value(model) = 19.0625
```

Рис. 23: Задание 1

Выполнение лабораторной работы

```
[104]: # 2. Линейное программирование. Использование массивов
```

```
vector_model = Model(GLPK.Optimizer)
```

```
[104]: A JuMP Model
      | solver: GLPK
      | objective_sense: FEASIBILITY_SENSE
      | num_variables: 0
      | num_constraints: 0
      | Names registered in the model: none
```

```
[105]: A = [-1 1 3; 1 3 -7]
      | b = [-5; 10]
      | c = [1; 2; 5]
```

```
[105]: 3-element Vector{Int64}:
      | 1
      | 2
      | 5
```

```
[107]: @variables(vector_model, begin
      |         x[1:3]>=0
      |     end)
```

```
[107]: (VariableRef{x[1]}, x[2], x[3]),)
```

```
[108]: @constraint(vector_model, A * x .<= b)
      | @constraint(vector_model, x[1] <= 10)
      | @objective(vector_model, Min, c' * x)
```

```
[108]:  $x_1 + 2x_2 + 5x_3$ 
```

Рис. 24: Задание 2

Выполнение лабораторной работы

```
[109]: optimize!(vector_model)
      termination_status(vector_model)

[109]: OPTIMAL::TerminationStatusCode = 1

[112]: @show value(x1);
      @show value(x2);
      @show value(x3);
      @show objective_value(vector_model);

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375
objective_value(vector_model) = 5.0
```

Рис. 25: Задание 2

Выполнение лабораторной работы

```
[113]: # 3. Векторное программирование

n = 3
m = 4
A = rand(1:100, m, n);
b = rand(1:100, m);

[114]: x = Variable(n)
problem = minimize(sumsquares(A*x-b))
constraints = [x>=0]
solve!(problem, SCS.Optimizer)

[ Info: [Convex.jl] Compilation finished: 3.93 seconds, 229.697 MiB of memory allocated

-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 4, constraints m: 6
cones:  q: soc vars: 6, qsize: 1
settings: eps_obj: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
         alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
         max_iters: 100000, normalize: 1, rho_x: 1.00e-06
         acceleration lookback: 10, acceleration interval: 10
         compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
         nnz(A): 14, nnz(P): 0
-----
iter | pri res | dual res | gap | obj | scale | time (s)
-----
0 | 1.57e+03 | 1.00e+00 | 2.22e+03 | -1.10e+03 | 1.00e-01 | 1.80e-03
250 | 1.35e+01 | 9.34e-02 | 1.42e+01 | 2.81e+03 | 1.02e+00 | 3.12e-02
500 | 1.48e+02 | 5.99e-01 | 3.83e+01 | 6.26e+03 | 3.18e-01 | 6.07e-02
625 | 1.69e-03 | 1.92e-05 | 2.15e-02 | 3.94e+03 | 3.18e-01 | 7.54e-02
-----
status: solved
timings: total: 7.54e-02s = setup: 9.49e-05s + solve: 7.53e-02s
         lin-sys: 2.96e-04s, cones: 1.62e-04s, accel: 2.47e-04s
-----
objective = 3936.604400
-----
```

Рис. 26: Задание 3

```
[115]: problem.optval  
[115]: 3936.5936487828303
```

Рис. 27: Задание 3

Выполнение лабораторной работы

```
[124]: # 4. Оптимальная раскладка по заказ
      priorities = [shuffle([1, 2, 3, 10000, 10000]) for _ in 1:1000]
      priorities = hcat(priorities...)'
      priority_weights = sum(priorities, dims = 1)

[124]: 1x5 Matrix{Int64}:
      3731238  3961223  3921266  4441113  3951160

[119]: modell = Model{GLPK.Optimizer};

[120]: @variable(modell, 180<=t1<=250)
      @variable(modell, 180<=t2<=250)
      @variable(modell, t3 == 220)
      @variable(modell, 180<=t4<=250)
      @variable(modell, 180<=t5<=250)

[120]: #5

[121]: @constraint(modell, t1+t2+t3+t4+t5==1000)

      @objective(modell, Min, t1 * priority_weights[1] +
      t2 * priority_weights[2] +
      t3 * priority_weights[3] +
      t4 * priority_weights[4] +
      t5 * priority_weights[5])

[121]: 381127761 + 412118662 + 384122063 + 408117464 + 415114365
```

Рис. 28: Задание 4

Выполнение лабораторной работы

```
[122]: optimize!(model1)
      termination_status(model1)

[122]: OPTIMAL::TerminationStatusCode = 1

[123]: @show value(t1);
      @show value(t2);
      @show value(t3);
      @show value(t4);
      @show value(t5);

      @show objective_value(model1);

value(t1) = 200.0
value(t2) = 180.0
value(t3) = 220.0
value(t4) = 180.0
value(t5) = 180.0
objective_value(model1) = 3.98340542e9
```

Рис. 29: Задание 4

Выполнение лабораторной работы

```
[150]: # 5. План приготовления кофе
      coffee = ["Raf", "Cappuccino"]
      products = ["Grains", "Milk", "Sugar"]
      costs = JuMP.Containers.DenseAxisArray{400, 500}, coffee)

[150]: 1-dimensional DenseAxisArray{Int64,1,...} with index sets:
      Dimension 1, ["Raf", "Cappuccino"]
      And data, a 2-element Vector{Int64}:
      400
      500

[151]: coffee_data = JuMP.Containers.DenseAxisArray{
      [40 140 5;
       50 120 0],
      coffee,
      products};

[152]: store = JuMP.Containers.DenseAxisArray{500, 2000, 40}, products)
      coffee_data["Raf", "Grains"]
      store["Grains"]

[152]: 500

[156]: model_coffee = Model{GLPK.Optimizer};

      @variable(model_coffee, buy[coffee] >= 0)

      @objective(model_coffee, Max, sum(costs[c]*buy[c] for c in coffee))
      @constraint(model_coffee, [p in products], sum(coffee_data[c, p]*buy[c] for c in coffee) <= store[p])

      JuMP.optimize!(model_coffee)
      term_status = JuMP.termination_status(model_coffee)
      hcat(buy.data, JuMP.value.(buy.data))

[156]: 2x2 Matrix{AffExpr}:
      buy[Raf]      8
      buy[Cappuccino] 6
```

Рис. 30: Задание 5

В результате выполнения работы были освоены пакеты Julia для решения задач оптимизации.